

RAG Architecture Documentation

Official Guide to Retrieval-Augmented Generation, Embeddings, and Hybrid Search

1. RAG Architecture Principles

Retrieval-Augmented Generation (RAG) augments Large Language Model prompts with non-parametric external knowledge retrieved from vector indexes (e.g. pgvector) using embedding similarity.

```
# High level RAG pipeline step:  
# 1. Query Embedding: vector = embed_model.embed_query(user_query)  
# 2. Vector DB Search: retrieved_chunks = db.search(vector, top_k=5)  
# 3. Context Injection: prompt = build_prompt(context=retrieved_chunks, query=user_query)  
# 4. LLM Response: response = llm.generate(prompt)
```

2. Hybrid Search & Reciprocal Rank Fusion (RRF)

Hybrid search combines dense semantic vector search (cosine similarity) with sparse keyword search (BM25). Scores are normalized and merged using Reciprocal Rank Fusion for optimal precision.

```
def reciprocal_rank_fusion(vector_results, bm25_results, k=60):  
    rrf_scores = {}  
    for rank, doc in enumerate(vector_results):  
        rrf_scores[doc.id] = rrf_scores.get(doc.id, 0) + 1.0 / (k + rank + 1)  
    for rank, doc in enumerate(bm25_results):  
        rrf_scores[doc.id] = rrf_scores.get(doc.id, 0) + 1.0 / (k + rank + 1)  
    return sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
```